TEKNEST — Master Build Plan (TekPractice first)

One organized home in D:, agents/identities, and the phased build with paste-ready Claude Code prompts. Focus: build TekPractice A→Z, live at teknest.fr, before touching the theoretical site.

Decisions locked

- **Home:** everything under `D:\teknest\`.
- **Hosting now:** TekPractice → **teknest.fr root** (theoretical site paused; game IS the site for now).
- **Milestone:** the **full vision** (Levels + Tests + Sandbox + Coach + Stats) — built in phases, each shippable.
- **Fable weekend:** build the 4 hard cores — Sandbox engine, Progression brain, remaining domain logic, 3D assets + white scene.

Target directory structure

```
D:\teknest\
  tekpractice\      ← the game (was C:\dev\audio-sim, GitHub: Technest-Sim)
  tekskol\          ← the site, PAUSED/dormant (was C:\dev\tech-nest-web, GitHub: Technest)
  docs\             ← all vision/status docs live here (one place, no mixing)
  README.md         ← the map: what each folder is, links to docs
~/.claude/agents\   ← shared agents: designer.md, developer.md (global, both projects use
them)
```

Rule going forward: **one home (`D:\teknest`), one window per repo, never Documents.**

PROMPT 1 — Reorganize into D:\teknest (run FIRST)

RUN IN: a NEUTRAL window (e.g. `C:\Users\iB_K`) — NOT inside the folders being moved. Close the audio-sim & tech-nest-web windows first. · **SERVES:** Foundation (all beats).

```
Objectif : tout réorganiser proprement sous D:\teknest. Ne casse rien, garde tout
l'historique git.

0. Vérifie d'abord que D: est un disque INTERNE fixe (pas amovible/USB), NTFS, avec assez
d'espace. Si D: est un HDD lent ou amovible, ARRÊTE et préviens-moi (on choisira un autre
chemin). Ajoute une exclusion Windows Defender pour D:\teknest (dossier) avant de copier —
c'est ce qui a étranglé les builds dans Documents.

1. Crée D:\teknest\ (+ sous-dossiers docs\).
2. robocopy C:\dev\audio-sim → D:\teknest\tekpractice : TOUT y compris .git, SAUF
node_modules. Puis robocopy C:\dev\tech-nest-web → D:\teknest\tekskol : idem.
3. Copie tous les .md de vision/statut depuis C:
\Users\iB_K\Documents\Claude\Projects\Thechnest\ (VISION_TekPractice-Game.md,
VISION_TekPractice-StudentJourney.md, STATUS.md, TEKNEST-MASTER-BUILD.md, et les db_*.csv) →
D:\teknest\docs\.
4. Dans D:\teknest\tekpractice : npm install, puis prouve les gates : npm run
```

```
validate:catalog (vert), npm run build (passe), npm test (vert), npm run dev (sert un canvas
Babylon). Confirme que git log montre l'historique complet et que le remote pointe toujours
vers Technest-Sim.
5. Écris D:\teknest\README.md : la carte du projet (tekpractice = le jeu, tekskol = site en
pause, docs = specs), + la règle "un seul home D:\teknest, une fenêtre par repo, jamais
Documents".
6. NE SUPPRIME PAS encore C:\dev — renomme C:\dev\audio-sim → audio-sim_DEAD et C:\dev\tech-
nest-web → tech-nest-web_DEAD (backup gelé, impossible à utiliser par erreur).
7. Mets à jour ta mémoire : le projet vit maintenant à D:\teknest\tekpractice et D:
\teknest\tekskol.

Quand tout est vert dans D:\teknest\tekpractice, dis-le et STOP. Ne code rien d'autre.
```

Then: open a fresh window in `D:\teknest\tekpractice` (that becomes your permanent TekPractice window).

PROMPT 2 — "Where are we, A→Z"

RUN IN: the **TekPractice** window (`D:\teknest\tekpractice`), opened fresh after the move. · **SERVES:** orientation (know the ground before building).

```
Fais-moi l'état des lieux COMPLET de TekPractice, de A à Z, sans rien inventer — lis le repo
et git :

1. Ce qui EXISTE et marche : catalog (connecteurs/signaux/devices/rules), niveaux (a1–d1 :
lesquels complets vs stubs), engine (ConnectionGraph, RuleEvaluator, LevelRunner, device
state/controls), logic (phantom/gpio/mixMinus/clock : lesquels implémentés vs stubs), assets
3D (combien de .glb dans public/assets, lesquels manquent vs le catalog), tests (combien, ce
qu'ils couvrent), CI, déploiement.
2. Ce qui MANQUE pour la vision complète (lis D:\teknest\docs\VISION_TekPractice-Game.md et
VISION_TekPractice-StudentJourney.md) : dashboard/stats de readiness, mode Tests exam,
Sandbox, environnements, coach/mentor, design blanc, 2D motion.
3. Un tableau : Composant | État (fait/partiel/absent) | Preuve (fichier/test) | Prochaine
action.
4. L'URL de déploiement actuelle et si teknest.fr pointe bien dessus.
Termine par : les 3 prochaines actions les plus à fort levier. Ne code rien.
```

AGENTS & IDENTITIES

File: `~/.claude/agents/designer.md`

```
---
name: designer
description: Teknest design authority. Owns ONE design system across the whole product — the
white, calm, premium art direction, tokens, components, 2D motion, and the 3D scene look. Use
for any UI, theme, visual, art-direction, or motion work.
tools: Read, Write, Edit, Bash, Glob, Grep
model: opus
---
Tu es le Designer de Teknest. Ta mission : le meilleur langage visuel éducatif-premium —
blanc, calme, gallery-grade (Teenage Engineering × Apple × vrai studio), qui BAISSE l'anxiété
et rend le vrai matériel héroïque.

Source de vérité : D:\teknest\docs\VISION_TekPractice-Game.md (§6 art direction) et
VISION_TekPractice-StudentJourney.md.
```

```
Règles :
- UN seul design system, partagé site+jeu. Tout est TOKEN (couleur, type, espacement, motion, élévation) — jamais de valeur en dur.
- Blanc dominant, un seul accent, whitespace généreux, typographie soignée, motion subtile.
- 2D motion = feedback pédagogique : VU-mètres, pulses de signal-flow, snap de câble, transitions douces — jamais gratuit.
- Scène 3D : sol blanc, ombres de contact douces, câbles caténaires élégants, le matériel est la star.
- Accessibilité WCAG AA. Toujours vérifier visuellement (screenshots) tes rendus.
Méthode : brainstorming → tokens (un fichier) → theme-factory pour appliquer partout → brand-guidelines → accessibility-review. Documente dans docs/design-system.md.
Garde-fou : ne casse pas la séparation des couches ; le contenu reste data. Premium et digne de confiance, jamais "jouet".
```

File: `~/.claude/agents/developer.md`

```
---
name: developer
description: Teknest engineering authority. Drills both codebases better and better — architecture, logic, tests, performance, the 3-layer/anti-drift discipline. Use for engine, logic, data model, tests, perf, CI, deploy.
tools: Read, Write, Edit, Bash, Glob, Grep
model: opus
---
Tu es le Developer de Teknest. Ta mission : une base propre, rapide, testée, qui rend la vision réelle sans dette.

Source de vérité : les CLAUDE.md des repos + D:\teknest\docs\VISION_*.md.

Règles :
- Respecte l'archi 3 couches : Content (data pure) / Engine (Babylon+rules) / Assets (glTF). Nouveau contenu = édition data, jamais de changement engine.
- Anti-drift : catalog = source de vérité, ne jamais réinventer.
- Invariants engine R1/R2/R3 toujours actifs ; logicChecks = modules R4–R8 seulement.
- Tests d'abord quand raisonnable ; chaque rule un cas triggers + un doesn't-trigger ; deep-scan avant chaque ship.
- Perf : glTF instanciés/lazy (Draco/KTX2), presets d'environnement légers, LOD, cap devices, fallback 2D basse-fidélité pour PC faibles, code-split.
- Ne jamais inventer un fait référentiel. git + backup GitHub. C:\... interdit — on vit dans D:\teknest.
Méthode : subagent-driven-development pour paralléliser, expert-council pour les forks durs, ADR pour toute décision non évidente. Arrête-toi aux gates.
```

Identities (headers to add to each repo's CLAUDE.md)

- **TekPractice** — top of `D:\teknest\tekpractice\CLAUDE.md`:
  `# IDENTITÉ : TekPractice — le jeu éducatif "reactive smart sound simulation" de Teknest. Mission : faire réussir le BTS Son (théorie + pratique) via un jeu blanc, calme, sans limites mais borné par le catalog réel. Délègue le visuel → agent designer, l'ingénierie profonde → agent developer. Vision : D:\teknest\docs\VISION_TekPractice-*.md.`
- **Tekskol (dormant)** — top of `D:\teknest\tekskol\CLAUDE.md`: `# IDENTITÉ : Tekskol — le site/école théorique de Teknest. EN PAUSE : ne pas développer tant que TekPractice n'est pas validé. Ne rien toucher ici sans instruction explicite.`

PROMPT 3 — Install the agents (run once)

RUN IN: the **TekPractice** window (it writes global `~/.claude/agents` + edits both CLAUDE.md files). · **SERVES:** all beats (the team that builds them).

```
Crée les deux fichiers d'agents globaux ~/.claude/agents/designer.md et ~/.claude/agents/
developer.md avec EXACTEMENT le contenu que je te donne [colle les deux blocs ci-dessus].
Puis ajoute l'en-tête d'IDENTITÉ en haut de D:\teknest\tekpractice\CLAUDE.md (TekPractice) et
de D:\teknest\tekskol\CLAUDE.md (Tekskol dormant). Confirme que `/agents` liste bien designer
et developer. Commit dans chaque repo.
```

Usage after: in the TekPractice window, delegate — e.g. "designer: applique le design system blanc au dashboard" or "developer: implémente le ProgressStore de readiness". The window is TekPractice; it calls the specialists.

NORTH-STAR — the 6 student beats drive every phase (the acceptance lens)

Every phase is validated by ONE question: **does it deliver a beat of Nadia's journey?** (docs/VISION_TekPractice-StudentJourney.md). Build for the beat, not the feature — a phase isn't "done" when the code passes, but when the beat *feels right for a real student*.

Beat	The student moment	Delivered by
1 — First 5 min	Calm white page → a real win in 90s (guided taste), no signup wall	Phase 1 (onboarding + white skin) + Phase 3 (guided-sandbox taste)
2 — Daily loop	ONE next action, micro-lesson, first level, mistakes teach, streak, Oscar's tip	Phase 1 (brain + dashboard) + Phase 2 (Learn/Levels) + Phase 5 (tips)
3 — Mastery, sandbox opens	Freedom unlocks; builds & shares a rig in any room	Phase 3 (sandbox) + Phase 4 (environments)
4 — The dip	Streak forgiven, an easier win, the mentor arrives at the low point	Phase 1 (streak-forgiveness) + Phase 5 (coach/psychology)
5 — Final stretch	Exam-mode tests → precise readiness report → a last-mile plan	Phase 2 (Tests) + Phase 1 (recommender)
6 — Exam day & after	Transfer: her hands already know it; she becomes a mentor voice	whole-design goal + later community

Each build prompt must state which BEAT it serves, and each gate is checked against that beat.

THE PHASES (full vision, each ships something)

Phase	What ships	Owner	Fable?
0 — Foundation	D:\teknest reorg, agents installed, status known, teknest.fr wired to TekPractice	Developer	no
1 — White skin + Dashboard + Readiness	A1–D1 playable in the white art direction; home dashboard with readiness radar + "one next action"; ProgressStore extended to référentiel-mapped stats + mistake history	Designer + Developer	brain = Fable
2 — Modes spine	Learn (light) + Levels (polished) + Tests (exam mode, no hints, scored vs référentiel) sharing the progression brain	Developer	no

Phase	What ships	Owner	Fable?
3 — Sandbox v1	Free-graph mode + category shelves + grab/drop/spawn + environment presets (studio, radio, plateau) + guided-sandbox gating; play feeds analytics	Developer	**Fable (hardest)**
4 — Environments + 2D motion + assets	théâtre / open-air / ENG / événementiel presets; VU-mètres, signal-flow pulses, cable feedback, UI transitions; all real 3D assets in	Designer + Developer	assets = Blender/Fable nights
5 — Coach + mentor	Oscar's context-timed tips + psychology layer (streak-forgiveness, adaptive ease, tone). AI coach later (ai-integrate)	Developer	optional
6 — Polish/perf/validate	budgets, LOD, 2D low-end fallback, deep-scan, tests; smooth on teknest.fr	Developer	no

Each phase ends **playable on teknest.fr** (the Friday no-drift rule).

FABLE WEEKEND SEQUENCING (scarce — spend on the hard cores)

- **Day (Fable, attended):** Phase 3 **Sandbox engine** + Phase 1 **Progression/readiness brain** — the two hardest reasoning systems. These are where Fable's lead is largest.
- **Night (Blender, unattended):** Phase 4 **3D asset generation** (all catalog devices) — pure grind, self-verifying, commits per model.
- **Quick (Fable or Opus):** finish/verify remaining domain logic (Phase 2) — small.
- **BEFORE Fable ends — the Durability Pass:** have Fable write specs + ADRs + paste-ready prompts for every phase it did NOT finish (1 skin/dashboard, 2 modes, 4 motion, 5 coach, 6 polish), so Opus/Sonnet execute Fable's plan next week. Its intelligence must end in FILES.

PROMPT 4 — Phase 1 core: Progression brain + Dashboard (Fable, day)

RUN IN: the **TekPractice** window (Fable model). · **SERVES BEATS 2, 4, 5** (one-next-action daily loop, streak-forgiveness at the dip, the readiness report).

```
Respecte CLAUDE.md + l'identité TekPractice. Phase 1 — le CERVEAU + le tableau de bord.

1. Étends ProgressStore (interface swappable, impl localStorage) en un modèle de READINESS
mappé au référentiel : par bloc (BC01–BC04) et par épreuve (E3/E4/E51), + historique
d'erreurs typé (R1–R8), + calcul d'un score de readiness par domaine, + un moteur "prochaine
meilleure action" (recommande l'activité qui attaque le point faible).
2. Dashboard d'accueil : radar de readiness, UNE action recommandée, streak (avec pardon :
rate un jour ne casse pas tout), tip contextuel.
3. Le mode Levels alimente ce modèle (chaque erreur/réussite met à jour la readiness).
Tests : le recommender choisit bien le domaine le plus faible ; la readiness monte quand on
réussit ; streak-forgiveness marche. ADR pour le modèle de readiness. Délègue le visuel à
l'agent designer (design system blanc). Arrête-toi au gate : dashboard réel + readiness qui
bouge, déployé.
```

PROMPT 5 — Phase 3 core: Sandbox engine (Fable, the hardest)

RUN IN: the **TekPractice** window (Fable). Not in parallel with Prompt 6 — same repo, one job at a time. · **SERVES BEAT 3** (mastery unlocks freedom; build & share a rig in any room).

```
Respecte CLAUDE.md + identité TekPractice. Phase 3 — la SANDBOX (cœur "sans limites"). Édite
content/ + une nouvelle couche sandbox, engine intact autant que possible ; toute extension =
ADR.

1. Mode free-graph : un "niveau" sans requiredChain — le joueur câble librement, MAIS le même
RuleEvaluator (R1–R8) valide et enseigne en direct.
2. Category shelves : UI d'étagères par catégorie (Micros, Consoles, Monitors, Stageboxes/IO,
Codecs, Patchbays, Câbles) lisant le catalog ; grab → drop → spawn du device dans la scène.
3. Environnements = presets data (backdrop + brief + sous-ensemble de matériel) : studio,
radio, plateau pour v1. Nouveau environnement = data, pas de code engine.
4. Guided-sandbox gating : palette réduite + un brief au début ; palette complète débloquée
par la maîtrise (lue depuis ProgressStore).
5. La sandbox ÉCRIT dans le modèle de readiness (le jeu libre = évaluation déguisée).
Sauvegarde/nommage d'un rig. Tests : free-graph valide via ConnectionGraph ; spawn depuis
shelf ; un environnement se charge par data seule. ADR pour toute extension d'interface. Run
long autonome. Gate : construire un rig dans un environnement, règles en direct, sauvegardé,
readiness mise à jour.
```

PROMPT 6 — Assets (Blender, night, unattended)

RUN IN: the **TekPractice** window (night; Blender open + MCP connected + never-sleep). Same repo as Prompts 4/5 → run when they're idle. · **SERVES BEATS 1 & 3** (the calm white world made real with real gear). (Use the asset prompt already validated: reads catalog.json for exact device ids + port_, white-scene-ready, .glb → gltf-transform, commit/push per model, skip-on-error, ASSET_MANIFEST.json. Run only with Blender open + MCP connected + Windows never-sleep.)

PROMPT 7 — Durability Pass (Fable, before it's gone — do NOT skip)

RUN IN: the **TekPractice** window (Fable, Sunday). · **SERVES:** continuity — Fable writes the specs + prompts so Opus finishes Beats 2, 4, 5, 6 next week.

```
CRITIQUE : après ce week-end je perds Fable 5. Rends-toi inutile — mets toute ton
intelligence sur disque pour qu'un modèle plus faible finisse.
Produis dans D:\teknest\tekpractice\docs\ :
1. ARCHITECTURE.md complet (couches, contrats, invariants, pièges).
2. ADRs pour chaque décision (readiness, sandbox, environnements, motion).
3. ROADMAP.md : chaque phase restante (1 skin/dashboard finition, 2 modes/Tests, 4
environnements+2D motion, 5 coach, 6 polish) avec but, fichiers, critères d'acceptation,
comment vérifier.
4. prompts/ : un prompt prêt-à-coller par phase restante (comme ceux qui ont construit ce
projet).
5. Durcis les tests : chaque rule, chaque invariant, chaque win-condition, un smoke par
niveau/mode.
Vérifie que tout est commité et vert. Puis un briefing de passation : ce qui est solide, ce
qui est fragile, quoi faire lundi.
```

Order of operations (do this, in order)

1. **Prompt 1** — reorg to D:\teknest (verify gates in the new home).
2. Open fresh window in `D:\teknest\tekpractice`. **Prompt 2** — A→Z status.

3. **Prompt 3** — install Designer + Developer agents + identities.
4. **Prompt 4** (Fable, day) — progression brain + dashboard. **Prompt 6** (Blender, night) — assets.
5. **Prompt 5** (Fable) — sandbox engine.
6. **Prompt 7** (Fable, Sunday) — Durability Pass.
7. Next week (Opus): execute Fable's ROADMAP + prompts → Phases 2, 4, 5, 6 → validate on teknest.fr.

Guardrails

- One home: **D:\teknest**. One window per repo. Never Documents.
- teknest.fr = TekPractice for now (site paused). Add Defender exclusion for D:\teknest.
- Keep GitHub backups current (both repos). Watch Fable credits: assets by night, reasoning by day — not both heavy at 3am.